وزارة التعليم العالي والبحث العلمي
MINISTRY OF HIGHER EDUCATION AND SCIENTIFIC RESEARCH

الجامعة السورية الخاصة
SYRIAN PRIVATE UNIVERSITY

Developing traditional Intrusion Detection System For Detecting Data Exfiltration tunnels

Supervisor: Waseem Juneidi & Ahmad Shekha

Students: Mohammad Al-Fahham & Sami Al-Fahel

Table of Contents

List of Figures 4
List of Table 4
List of Abbreviations 5
1.Introduction 7
1.1.Background : 7
1.2. Problem Statement 8
1.3. Project Goals 9
1.4. Application Domains 11
1.4.1 Public and Private Cloud Environments 11
1.4.2 On-Premise, Containerized, and Hybrid Infrastructure 11
1.4.3 Enterprise Security Operations Centers (SOCs) 12
1.4.4 Defense Against Supply Chain and Command-and-Control Attacks 12
1.4.5 IoT and Resource-Constrained Network Environments 13
1.4.6 Organizations Handling Sensitive or Regulated Data 13
1.5. Anticipated Challenges 13
1.5.1. ICMP Tunneling Has No Established Detection Baseline 13
1.5.2. Encrypted Tunneling Traffic Limits Content-Based Detection 14
1.5.3. Ambiguity in DNS Record Type Signatures 14
1.5.4. Risk of Degraded Accuracy from Rule Set Expansion 15
1.5.5. Snort's Sensitivity to Throughput in Cloud-Like Environments 15
1.5.6. Small Sample Sizes Limit Statistical Confidence in the Baseline 15
1.5.7. Balancing Improved Recall Against False Positive Rates 16
1.5.8. Generalizability Beyond the Original AWS Environment 16
1.5.9. Lack of Standardized Testing Frameworks 17
1.5.10. Continuous Evolution of Tunneling Techniques 17
1.6. Related Work (Literature Review) 17
1.6.1. Machine Learning and Explainable AI for ICMP Detection 17
1.6.2. Multi-Layer Steganography and Structural Analysis 18

1.6.3. NIDS Benchmarking and Comparative Evaluations 18
1.6.4. Thematic Summary Matrix 19
1.6.5. Research Gap and Project Alignment 19
2. Chapter 1: Theoretical Background and Literature Review 20
2.1. Computer Networks Components, Architecture, and Vulnerabilities 20
2.1.1. Major Components of Computer Networks 20
2.1.2. Network Architectures: Common Models and Their Characteristics 21
2.1.3. Vulnerabilities 22
2.2. Types of IDSs 22
2.3. IDSs Used 27
2.3.1. Snort 27
2.3.2. Suricata 28
2.4. Traffic Monitoring Mechanisms 30
2.4.1. Non-intrusive Monitoring Concept 30
2.4.2. On-Premise Traffic Capture Techniques: Network TAP vs. SPAN Port 31
2.4.3. Cloud Traffic Mirroring Mechanisms 32
2.4.4. Data Pipeline Architecture 32
2.4.5. Technical Monitoring Challenges: High Throughput and Encrypted Traffic 33
2.5. Test Instrumentation: The ELK Stack 34
2.6. Attack and Tunneling Mechanisms 36
2.6.1. DNS Tunneling: DNSCat2 and Iodine 36
2.6.2. ICMP Tunneling (Metasploit) : 37
2.7. Virtualization Technology 38
2.7.1. VMware Virtualization Platform 38
2.8. Evaluation Metrics 39
References 41

List of Figures

Figure 1: Schematic representation of the HIDS concept. 24
Figure 2: Schematic representation of both the anomaly-based and signature-based IDS concepts. 26
Figure 3: Snort Architecture 28
Figure 4: Suricata Architecture 30
Figure 5: Illustration of DNS Tunneling Used for C2 Communication. 37
Figure 6: Proposed Solution Roadmap. 41
Figure 7: Proposed Implementation Flowchart. 42

List of Table

Table 1: Thematic Organization of Literature on ICMP Tunneling Detection. 19

List of Abbreviations

#	Abbreviation	Full Form
1	IDS	Intrusion Detection System
2	DNS	Domain Name System
3	ICMP	Internet Control Message Protocol
4	DDoS	Distributed Denial of Service
5	AWS	Amazon Web Services
6	ELK	Elasticsearch, Logstash, Kibana
7	C2	Command and Control
8	IoT	Internet of Things
9	CNAME	Canonical Name Record
10	MX	Mail Exchanger Record
11	TXT	Text Record
12	NULL	Null Record
13	EC2	Elastic Compute Cloud
14	VPC	Virtual Private Cloud
15	IPS	Intrusion Prevention System
16	IP	Internet Protocol
17	LAN	Local Area Network
18	MAC	Media Access Control
19	SPAN	Switched Port Analyzer
20	AP	Access Point
21	P2P	Peer-to-Peer
22	TCP	Transmission Control Protocol
23	DPI	Deep Packet Inspection
24	NIDS	Network Intrusion Detection System
25	TLS	Transport Layer Security
26	SSL	Secure Sockets Layer
27	HIDS	Host Intrusion Detection System
28	APT	Advanced Persistent Threat
29	ML	Machine Learning
30	ICS	Industrial Control Systems
31	HTTP	Hypertext Transfer Protocol
32	NSM	Network Security Monitoring
33	JSON	JavaScript Object Notation
34	SIEM	Security Information and Event Management

#	Abbreviation	Full Form
35	YAML	Yet Another Markup Language
36	TAP	Test Access Point
37	RSPAN	Remote SPAN
38	VLAN	Virtual Local Area Network
39	ERSPAN	Encapsulated Remote SPAN
40	GRE	Generic Routing Encapsulation
41	STP	Spanning Tree Protocol
42	CDP	Cisco Discovery Protocol
43	CPU	Central Processing Unit
44	SDN	Software-Defined Networking
45	vNIC	Virtual Network Interface Card
46	ENI	Elastic Network Interface
47	VXLAN	Virtual Extensible LAN
48	KQL	Kibana Query Language
49	VM	Virtual Machine
50	VMM	Virtual Machine Monitor
51	NAT	Network Address Translation
52	IoC	Indicators of Compromise
53	IPRep	IP Reputation
54	MD5	Message Digest 5
55	TP	True Positive
56	FP	False Positive
57	FN	False Negative
58	TN	True Negative
59	TPR	True Positive Rate
60	KNN	K-Nearest Neighbors
61	SVM	Support Vector Machines

1.Introduction

1.1.Background :

Data exfiltration is one of the most pressing challenges in modern cybersecurity, since it involves the unauthorized transfer of sensitive information out of an organization's network, directly compromising confidentiality and integrity and exposing organizations to serious financial and reputational damage [1].

Intrusion Detection Systems (IDS) are a core defense against this threat, but their effectiveness drops sharply when attackers use covert channels such as DNS and ICMP tunneling rather than conventional exfiltration methods [1].

What makes these two techniques especially dangerous is the trust built into the protocols themselves.

DNS is essential to normal internet operation and is almost universally permitted through firewalls with minimal scrutiny, which makes it an attractive vector for attackers to smuggle data out disguised as routine queries [1].

ICMP carries the same advantage: because it's used for everyday network diagnostics like ping, it's typically treated as benign by default and rarely inspected closely, even though it can just as easily carry an encapsulated covert channel [1].

This isn't a theoretical concern. High-profile cyber-espionage incidents have used exactly this kind of tunneling to bypass traditional security measures, most notably the SolarWinds Orion supply chain attack, where DNS tunneling was used for command-and-control communication, letting attackers exfiltrate data through DNS queries that looked completely ordinary [2].

The scale of the problem is confirmed empirically in recent research.

A 2026 comparative evaluation of the two leading open-source IDS platforms, Snort and Suricata, tested their detection capabilities against DNS tunneling (via DNSCat2 and Iodine) and ICMP tunneling (via Metasploit) in a cloud environment [1].

The results exposed a real, measurable gap: both tools failed completely to detect ICMP tunneling under their default configurations, recording 0% recall, while Snort also struggled significantly with Iodine-based DNS tunneling, catching only 66.7% of attempts [1].

Even Suricata, the stronger performer of the two, only reached its near-perfect detection rates because of the specialized, hand-adapted rules used in that study, not out-of-the-box defaults.

Given that Snort and Suricata are among the most widely deployed IDS tools in production environments worldwide, this gap means a substantial number of real networks are currently blind to one of the most effective covert exfiltration techniques in use today, which is precisely the gap this project sets out to close.

1.2. Problem Statement

Despite their widespread deployment, both Snort and Suricata exhibit a critical weakness when faced with covert tunneling-based exfiltration.

The comparative evaluation in [1] showed that neither IDS could detect ICMP tunneling under default rules and configurations, registering 0% recall across all attempts using the Metasploit icmp_tunnel module.

The root cause is structural rather than incidental: default rule sets in both tools are built around well-known threats like malware, exploits, DDoS, and port scanning, while ICMP traffic itself is treated as inherently benign since it's essential for routine diagnostics such as ping [1].

As a result, neither tool scrutinizes ICMP payloads for the anomalies that actually indicate tunneling, such as unusually large or high-frequency packets, unless those rules are explicitly added.

Because both tools rely on signature-based detection rather than behavioral analysis by default, an attacker who hides data inside ICMP echo requests and replies passes through completely unnoticed [1].

The picture for DNS tunneling is less catastrophic but still exposes meaningful weaknesses, particularly in Snort.

Even with detection rules adapted specifically for DNSCat2 and Iodine, Snort missed one out of seven DNSCat2 attempts (85.7% recall) and, more significantly, one out of three Iodine attempts (66.7% recall) — meaning a third of Iodine-based tunneling activity went undetected even with tuned rules in place [1].

Suricata performed better across both tools, attributed to its multi-threaded, protocol-aware architecture being better suited to the high-volume, virtualized traffic patterns typical of cloud environments, where Snort's rule evaluation is more sensitive to throughput constraints [1].

Taken together, the paper frames this as a problem that current commercial-grade IDS tools cannot solve out of the box: ICMP tunneling is essentially invisible to default configurations, and even purpose-built DNS tunneling rules leave detectable gaps, especially under Snort [1].

The authors explicitly call this an open problem requiring custom rule development, continuous tuning, and ultimately the integration of behavior-based or machine-learning-driven anomaly detection layered on top of traditional signature matching, since signature matching alone cannot keep pace with the variability of tunneling techniques [1].

This is the precise gap your project is positioned to close: building a detection layer that specifically targets the blind spot around ICMP tunneling and strengthens DNS tunneling detection beyond what default and even moderately tuned Snort/Suricata configurations currently achieve.

1.3. Project Goals

Building directly on the gap identified in [1], this project aims to design and implement a detection layer that closes the specific blind spots left by default Snort and Suricata configurations, with a primary focus on ICMP tunneling and a secondary focus on strengthening DNS tunneling detection.

The goals are defined as follows:

Goal 1: Close the ICMP tunneling detection gap. Since [1] showed that neither Snort nor Suricata detects ICMP tunneling under default configurations (0% recall across all tested attempts), the first and most critical objective of this project is to develop custom detection rules and/or behavioral analysis capable of identifying anomalous ICMP traffic, such as abnormally large payloads, high-frequency echo request/reply patterns, or other byte-level signatures indicative of tunneling activity.

This directly answers the call in [1] for "specialized rule development and tuning" to address ICMP tunneling, which the authors identify as a shared limitation of both tools.

Goal 2: Strengthen DNS tunneling detection beyond current Snort performance.

While Suricata achieved strong results against DNSCat2 and Iodine in [1], Snort's detection rate dropped to 66.7% recall against Iodine specifically, leaving a third of tunneling attempts undetected even with adapted rules.

This project aims to refine and extend the DNS detection ruleset to close this gap, ensuring consistent detection performance across tunneling tools rather than relying on Suricata's architectural advantage alone to carry overall system reliability.

Goal 3: Move beyond signature-only detection. The paper explicitly attributes the ICMP blind spot to an over-reliance on signature-based detection rather than behavioral analysis [1].

In line with this, a goal of this project is to incorporate behavior-based or anomaly-based detection logic, where traffic is flagged based on deviation from a normal baseline rather than requiring traffic to match a known static signature.

This is intended to make the system more resilient to evasion than the purely rule-matching approach evaluated in [1].

Goal 4: Validate detection performance using the same evaluation framework as the original study.

To allow direct, meaningful comparison against the baseline results in [1], this project will use the same evaluation metrics (Recall, Precision, Accuracy, F1-Score) and ideally the same or comparable attack tools (Metasploit's icmp_tunnel module, DNSCat2, Iodine), so that improvement over the original 0% ICMP recall and 66.7% Iodine recall can be quantitatively demonstrated rather than only claimed.

Goal 5: Maintain low false positive rates. The original study found that both Snort and Suricata achieved 0% false positives across all scenarios when properly configured [1].

Any new detection logic introduced in this project must preserve this specificity;

a system that closes the recall gap by generating excessive false alarms would not represent a genuine improvement, since the paper highlights low false-positive rates as one of the key strengths to preserve.

Collectively, these goals operationalize the future work explicitly proposed in [1] — custom rule development for ICMP tunneling, continuous tuning of DNS detection rules, and the integration of more advanced detection techniques beyond static signature matching.

1.4. Application Domains

The relevance of closing the DNS and ICMP tunneling detection gap is not confined to a single deployment scenario.

Because the underlying weakness — signature-based IDS tools failing against covert channels built on trusted protocols — is structural rather than environment-specific, the detection improvements this project develops have direct application across several real-world contexts.

1.4.1 Public and Private Cloud Environments

The most direct application domain is the one the original study itself was built around: cloud computing infrastructure.

Cloud environments have transformed IT resource management by offering unprecedented scalability, flexibility, and cost efficiency, which has driven their widespread adoption across virtually every industry [1].

However, this same shift has introduced security challenges that traditional on-premise tools were never designed to handle, particularly around virtualized networking, traffic mirroring, and the high-volume, multi-tenant traffic patterns typical of platforms like AWS [1].

Organizations migrating sensitive workloads to public cloud platforms are exposed to exactly the kind of DNS and ICMP tunneling techniques evaluated in [1], where data can be exfiltrated through protocols that cloud security groups and firewalls almost always permit by default.

A detection layer that closes this gap is directly applicable to any organization running IDS instances inside AWS, Azure, or Google Cloud environments using traffic mirroring architectures similar to the one described in the paper.

1.4.2 On-Premise, Containerized, and Hybrid Infrastructure

Although the original experiments were conducted entirely on AWS, the paper is explicit that this was a matter of convenience rather than necessity: Snort and Suricata are infrastructure-agnostic tools that can be deployed across on-premise, containerized, Kubernetes, and hybrid environments, and the detection behavior observed is expected to generalize to any infrastructure with comparable networking and traffic characteristics [1].

This means the detection improvements developed in this project are not limited to cloud-hosted systems.

They are equally applicable to traditional on-premise data centers, containerized microservice deployments running in Kubernetes clusters, and hybrid architectures that span both on-premise and cloud resources — all of which are exposed to the same DNS and ICMP tunneling risks, since these techniques target protocol-level trust rather than any specific hosting model.

1.4.3 Enterprise Security Operations Centers (SOCs)

A second application domain is the modern Security Operations Center, where centralized logging and visualization platforms like the ELK Stack (Elasticsearch, Logstash, Kibana) are used to aggregate IDS alerts for real-time monitoring, threat analysis, and incident response [1].

The paper integrates Snort and Suricata output directly into ELK via Filebeat to enable this kind of centralized visibility [1].

Any SOC running a similar logging pipeline can directly benefit from improved ICMP and DNS tunneling detection rules, since better detection at the IDS layer translates into more accurate, more complete alert data flowing into the dashboards analysts rely on for threat hunting and forensic investigation.

This is particularly valuable for compliance and reporting use cases, where the ELK Stack is also used to generate detailed logs of network activity and security events for regulatory purposes [1].

1.4.4 Defense Against Supply Chain and Command-and-Control Attacks

Beyond infrastructure type, this project has direct application in defending against a specific and highly consequential attack pattern: the use of DNS tunneling for command-and-control (C2) communication in supply chain attacks.

The paper cites the SolarWinds Orion attack as a real-world example, where DNS tunneling allowed attackers to exfiltrate data and maintain C2 channels through DNS queries that appeared completely benign to existing defenses [2].

This is not an isolated incident; several high-profile cyber-espionage operations have relied on covert DNS and ICMP tunneling specifically because these channels bypass traditional security controls [1].

Organizations in sectors that are common targets for nation-state or supply-chain-style attacks — government agencies, defense contractors, financial institutions, and large software vendors with extensive customer bases — stand to benefit directly from stronger tunneling detection, since these are exactly the actors who have historically been targeted using these techniques.

1.4.5 IoT and Resource-Constrained Network Environments

Related literature reviewed in the paper also points to the Internet of Things (IoT) as a domain where IDS tools like Snort are deployed to address resource constraints, varied communication protocols, and the need for tailored defense mechanisms [1].

While IoT environments were not the primary focus of the original study, the same DNS and ICMP tunneling techniques are applicable threats in IoT contexts, where devices frequently communicate using lightweight protocols that are rarely scrutinized as closely as conventional web traffic.

Improved detection rules developed in this project, particularly the lightweight signature and threshold-based rules used for Iodine and DNSCat2 detection [1], are well suited to extension into IoT and other resource-constrained network environments where heavier behavioral analysis tools may not be practical.

1.4.6 Organizations Handling Sensitive or Regulated Data

Finally, at a broader level, any organization whose primary security concern is protecting sensitive information — healthcare providers, financial services, educational institutions, and critical infrastructure operators — represents a relevant application domain for this work.

Data exfiltration fundamentally threatens confidentiality and integrity and exposes organizations to significant financial and reputational damage regardless of sector [1].

Since DNS and ICMP tunneling are protocol-level techniques rather than industry-specific exploits, the detection improvements developed here are broadly applicable to any organization where the unauthorized transfer of sensitive data — whether intellectual property, personal records, financial information, or operational data — represents a meaningful risk.

1.5. Anticipated Challenges

Closing the detection gap identified in [1] is not a straightforward engineering task of simply "adding more rules."

The paper itself, both directly and through the limitations it acknowledges in its own methodology, points to a number of difficulties that this project will need to confront.

These challenges span technical, methodological, and practical dimensions.

1.5.1. ICMP Tunneling Has No Established Detection Baseline

The most significant challenge this project faces is that, unlike DNS tunneling, there is no working detection baseline to build from.

The paper adapted its DNS tunneling rules from prior published research [3], giving it a tested starting point for DNSCat2 and Iodine detection.

For ICMP tunneling, however, no such baseline exists in the study;

both Snort and Suricata recorded a flat 0% recall, and the paper does not provide working rules that successfully detect it, only a diagnosis of why detection failed and a recommendation that "additional rules should be developed to detect anomalies, such as considerable ICMP payloads, high-frequency ICMP traffic, or specific byte patterns indicative of tunneling" [1].

This means this project must design ICMP detection logic essentially from first principles, with no validated reference implementation to adapt, replicate, or benchmark against.

Determining the right thresholds — how large is "too large" for an ICMP payload, how frequent is "too frequent" for echo requests — will require original experimentation rather than reproduction of an existing approach.

1.5.2. Encrypted Tunneling Traffic Limits Content-Based Detection

DNSCat2 encrypts its tunnel traffic by default, which means detection cannot rely on inspecting the actual content of the DNS payload [1].

Instead, the paper's detection rules work around this by relying on indirect, structural indicators: a plaintext 5-byte header and 16-bit nonce that remain visible even under encryption, the use of specific record types such as CNAME, MX, and TXT, and the fact that DNS query and reply packets exceed 100 bytes in size [1].

This is a meaningfully constrained detection surface. Any extension or refinement of these rules in this project will face the same limitation: behavioral and structural fingerprinting must substitute for direct content inspection, which makes the detection logic more fragile and more easily evaded if an attacker varies packet sizes, timing, or header structure to stay under the thresholds being used.

1.5.3. Ambiguity in DNS Record Type Signatures

A related challenge is specific to Iodine, which exploits the NULL DNS resource record type to create its covert channel [1].

While NULL records are uncommon in legitimate DNS traffic, the paper is explicit that their presence "alone is not always sufficient to conclusively identify malicious activity" [1].

This creates a precision–recall tension: rules based purely on the presence of a NULL record risk either being too narrow (missing variant tunneling behavior) or too broad (flagging legitimate, if unusual, DNS configurations as malicious).

Designing detection logic that distinguishes genuinely malicious NULL record usage from rare-but-legitimate use cases is a non-trivial challenge this project will need to address directly.

1.5.4. Risk of Degraded Accuracy from Rule Set Expansion

The paper's [1] review of related work raises a specific and directly relevant caution: research on Suricata has shown that detection accuracy notably decreases as the number of active rules increases, with significant performance drops beginning around 100,000 rules in isolated scenarios, and degradation appearing even earlier, past 10,000 rules per indicator, in combined scenarios [4].

This is directly relevant to this project's goal of expanding detection coverage for both ICMP and DNS tunneling: simply adding more and more custom rules to close detection gaps is not a free strategy.

Every additional rule introduces processing overhead and potential rule-interaction effects that can degrade overall detection accuracy, meaning rule additions in this project will need to be deliberate and tested for their impact on the overall ruleset's performance, not just evaluated in isolation.

1.5.5. Snort's Sensitivity to Throughput in Cloud-Like Environments

The paper attributes part of Snort's weaker performance against Iodine specifically to architectural constraints: Snort's rule evaluation is more sensitive to throughput limitations than Suricata's multi-threaded, protocol-aware design, particularly under the high-volume, virtualized traffic conditions typical of cloud environments [1].

This presents a challenge for any improvements made specifically to Snort rules in this project — even well-designed detection logic may underperform in practice if the underlying single-threaded (or less parallelized) processing pipeline cannot keep pace with traffic volume, independent of how good the rule itself is.

Any meaningful improvement to Snort-side detection may therefore need to consider performance and architectural constraints alongside rule logic, rather than treating detection accuracy as a purely rule-writing problem.

1.5.6. Small Sample Sizes Limit Statistical Confidence in the Baseline

A methodological challenge inherited from the original study is the limited scale of its test scenarios: only 3 ICMP tunneling attempts, 7 DNSCat2 attempts, and 3 Iodine attempts were tested [1].

The paper acknowledges this limitation itself, calling for future work involving "more tunneling scenarios and repeated trials to enable statistically meaningful analysis" [1].

This means the baseline numbers this project is working to improve upon — 0% ICMP recall, 85.7% DNSCat2 recall, 66.7% Iodine recall — are drawn from a small number of trials and may not be fully representative of real-world detection rates under varied conditions.

Replicating or improving on these results in a way that is statistically credible will require running a larger number of trials than the original study did, which has direct implications for the time and infrastructure this project will need.

1.5.7. Balancing Improved Recall Against False Positive Rates

The original study found zero false positives across all tested scenarios for both Snort and Suricata [1], which the paper highlights as a key strength worth preserving.

This creates a real constraint on how aggressively detection rules can be tuned: rules that are loosened or broadened to catch more true positives — particularly for ICMP tunneling, where no working detection exists yet — risk introducing false positives that did not exist in the baseline.

A solution that "solves" the ICMP detection gap by flooding analysts with false alerts would not represent genuine improvement, since excessive false positives undermine the practical usability of an IDS regardless of its raw detection rate.

Striking this balance, especially for a technique with no prior working detection rules to calibrate against, will likely require iterative tuning and testing rather than a one-shot rule deployment.

1.5.8. Generalizability Beyond the Original AWS Environment

While the paper argues that Snort and Suricata are infrastructure-agnostic and that its findings should generalize to on-premise, containerized, Kubernetes, and hybrid environments with comparable networking characteristics [1], this claim was not actually tested empirically;

all experiments were conducted on AWS EC2 instances using AWS-specific VPC Traffic Mirroring [1].

This presents a challenge if this project's testbed differs from the original AWS-based setup — whether due to budget constraints, access limitations, or a deliberate choice to build a local or on-premise testbed instead.

Traffic mirroring, packet capture mechanisms, and virtualized networking behavior can differ meaningfully between cloud providers and on-premise setups, meaning detection rules and thresholds tuned in one environment may need re-validation rather than direct transfer if this project's test environment doesn't closely mirror the original AWS architecture.

1.5.9. Lack of Standardized Testing Frameworks

The paper closes by noting a structural problem affecting this entire line of research: the absence of standardized testing frameworks for IDS/IPS platforms, combined with a wide variety of available rule libraries and a rapidly evolving threat landscape [1].

This makes it difficult to establish clear, agreed-upon performance boundaries for comparison, even when using the same evaluation metrics (Recall, Precision, Accuracy, F1-Score).

For this project, this means there is no single authoritative way to prove that a new rule or detection method is definitively "better" beyond comparing it against the specific baseline numbers in [1] — broader claims about general IDS improvement would need to be made cautiously, since the field itself lacks the standardized benchmarks that would make such claims fully rigorous.

1.5.10. Continuous Evolution of Tunneling Techniques

Finally, the paper frames its own results as a snapshot rather than a permanent verdict, explicitly calling for "continuous tuning" since rule sets must be regularly updated to adapt to evolving tunneling techniques [1].

Tools like DNSCat2 and Iodine, as well as ICMP tunneling implementations such as Metasploit's icmp_tunnel module, are actively maintained and can change behavior across versions.

A detection rule tuned precisely against the version and configuration used in the original study may not perform identically against a newer release or a deliberately modified variant designed to evade known signatures.

This means any detection logic developed in this project should be treated as a living component requiring ongoing maintenance, not a one-time deliverable — a reality worth acknowledging explicitly in the report's limitations and future work sections.

1.6. Related Work (Literature Review)

1.6.1. Machine Learning and Explainable AI for ICMP Detection

- Phan and Jerabek [5] developed an explainable AI (XAI) framework integrated with machine learning models to identify evasive ICMP and IPv6 covert channels. Their methodology evaluates dynamic packet characteristics and provides transparent decision-making logic for security analysts encountering sophisticated data exfiltration.
- Kaddissi et al. [6] introduced the ICMP Covert Channel Ranking System (ICMP C2RS), a real-time risk evaluation framework designed to score and detect covert data exfiltration via ICMP Echo payload alterations. Their system demonstrates high real-time throughput with minimal classification latency.
- Lipsa et al. [7] presented an interpretable dimensionality reduction technique paired with an explainable intrusion detection architecture for ICMP tunneling. The

model isolates subtle payload anomalies and payload length variations, achieving high detection precision against obfuscated channels.

- Ahmad et al. [8] provided a comprehensive evaluation of deep learning architectures applied to network covert channel detection, demonstrating that recurrent neural network (RNN) variants significantly outperform traditional thresholding in identifying low-rate ICMP timing and storage channels.

1.6.2. Multi-Layer Steganography and Structural Analysis

- Al-Ahmad and Huda [9] proposed a multi-layer network steganography framework combining ICMP covert channels with NTFS Alternative Data Streams (ADS). Their research highlighted how hybrid covert techniques bypass perimeter firewalls by embedding sensitive payloads across multiple abstraction layers.
- Merezas and Wüstrich [10] conducted a detailed structural evaluation of ICMP packet architecture under RFC 792, demonstrating how unstandardized payload fields in Echo Request (Type 8) and Echo Reply (Type 0) packets facilitate the covert tunneling of full TCP/UDP connections and Command-and-Control (C2) botnet traffic (such as the Pingback backdoor). Their study underscored stateful packet tracking and payload normalization as critical, low-latency countermeasures.
- Kumar and Singh [11] formulated a stateless scrubbing defense mechanism that inspects and sanitizes ICMP Echo Request payloads in real time. By zeroing out non-standard data fields without affecting core network diagnostic commands, their approach neutralizes exfiltration attempts at the network layer.
- Liu et al. [12] introduced a control-plane analysis framework for recognizing anomalous ICMP traffic patterns. Rather than relying on heavy machine learning training, their scheme combines multi-feature entropy metrics to reveal covert ICMP behavior that remains invisible during single-parameter checks.

- Wang et al. [13] engineered a baseline feature extraction model capable of sub-millisecond ICMP packet parsing. Their system establishes dynamic baseline metrics for standard diagnostic traffic, triggering instant alerts when payload frequency or volume diverges from established norms.

1.6.3. NIDS Benchmarking and Comparative Evaluations

- Qutqut et al. [1] conducted an empirical comparative evaluation of signature-based and multi-threaded Network Intrusion Detection Systems (Snort and Suricata) against automated data exfiltration tools. Their experiments confirmed that default rule configurations in both Snort and Suricata exhibit a 0% recall rate when

detecting ICMP tunnels generated by frameworks such as Metasploit, emphasizing the critical need for custom operational signatures.

- Chen et al. [14] developed "CC-Guard", a specialized detection system designed for ICMPv6 covert channels. By employing real-time field-matching algorithms across IPv6 structures, CC-Guard accurately detects multi-vector covert traffic in modern enterprise networks.

1.6.4. Thematic Summary Matrix

To synthesize the existing body of knowledge, the reviewed literature is categorized into core research themes in Table 1.

Table 1: Thematic Organization of Literature on ICMP Tunneling Detection.

Research Theme	Key References	Core Insights & Academic Findings
Machine Learning & XAI Detection	[5,6,7,8]	Modern ML and Explainable AI (XAI) models offer high accuracy and transparent decision-making for isolating obfuscated ICMP payloads and timing anomalies.
Structural Analysis & Protocol Exploitation	[9,10,11,12,13]	Hybrid attacks combine ICMP with file-system steganography. In response, payload scrubbing and entropy analysis provide lightweight, low-latency defenses without diagnostic interruption.
NIDS Performance & Security Gaps	[1,14]	Out-of-the-box Snort and Suricata rulesets fail completely (0% recall) against ICMP exfiltration tools, driving the requirement for optimized, custom NIDS signatures

1.6.5. Research Gap and Project Alignment

Despite recent advancements in machine learning detection models (2025–2026), the literature highlights a persistent operational vulnerability: widely deployed Network Intrusion Detection Systems (NIDS) like Snort and Suricata fail to detect ICMP tunneling out-of-the-box, showing a 0% recall rate under default rule sets. Existing academic solutions often require significant computational overhead for deep learning inference or focus solely on theoretical payload scrubbing, lacking ready-to-deploy signature configurations for production environments.

This project directly addresses this gap by engineering and validating optimized, low-overhead custom rules for both Snort and Suricata. By bridging the gap between theoretical anomaly metrics (such as payload size variance and request frequency) and practical NIDS rule engineering, this work delivers a production-ready defense mechanism capable of intercepting ICMP-based data exfiltration in real-time.

2. Chapter 1: Theoretical Background and Literature Review

2.1. Computer Networks Components, Architecture, and Vulnerabilities

Computer networks constitute the fundamental infrastructure for modern technologies, utilized for communication, resource sharing, and running critical applications.

Understanding the main components and common architectures of these networks is essential for designing efficient, secure, and scalable systems, particularly when seeking to protect them against advanced threats such as data exfiltration via covert tunneling attacks.

2.1.1. Major Components of Computer Networks

A computer network is made up of an integrated combination of hardware and software, each playing a specific role in ensuring connectivity and overall operational functionality.

Key hardware components include devices such as routers, switches, modems, and access points, which act as the backbone of the network.

Routers: A router is a crucial device that connects different networks and routes data traffic based on IP addresses, ensuring communication between separate subnets and the Internet.

They often feature built-in security mechanisms such as firewalls.

In the context of this project, routers and cloud gateways represent the critical egress points and final lines of defense that attackers exploit to route data exfiltration tunnels toward malicious external servers.

Switches: On the other hand, switches focus on connecting devices within a single local area network (LAN) at the data link layer using MAC addresses, thereby reducing congestion and increasing network efficiency compared to traditional hubs.

The significance of switches in this research lies in their strategic positioning for implementing non-intrusive monitoring techniques.

Specifically, port mirroring (SPAN) is configured at this layer to duplicate live traffic and feed it into Intrusion Detection Systems (IDS) without impacting network throughput or introducing latency [15].

Wireless Access Points (APs): These devices allow endpoints to connect to the network without physical cabling, offering considerable flexibility despite being susceptible to interference and general performance limitations compared to wired connections.

Terminals / Endpoints: Computers, smartphones, servers, and Internet of Things (IoT) devices serve as the network's endpoints where data is consumed or generated.

Within this project, these endpoints constitute the initial point of compromise.

Adversaries target these devices to deploy covert tunneling tools (such as DNSCat2 or Iodine), effectively transforming a legitimate node into a client that exfiltrates the organization's sensitive assets.

2.1.2. Network Architectures: Common Models and Their Characteristics

Network architectures define the logical and physical structure governing how devices interact and exchange data.

Different models are deployed based on specific operational needs, each carrying distinct advantages and limitations.

Peer-to-Peer (P2P): In this model, all connected nodes act simultaneously as both clients and servers, making it ideal for small environments but highly complex to manage and secure as the network scales.

Client–Server Architecture: This is the most prevalent choice in enterprise environments, where one or more central servers manage requests from client devices to provide centralized access to resources, improving security but introducing a single point of failure.

This architecture serves as the primary host for vital internet protocols, such as the Domain Name System (DNS).

Because of the inherent trust embedded in the client-server relationship, adversaries exploit this mechanism to establish DNS tunnels.

Consequently, malicious traffic masquerades as legitimate lookup requests sent from a client to a server, while masking unauthorized data exfiltration.

Network Topologies: Physical configurations range from star topologies, which connect all devices to a central node like a switch to ensure fault tolerance, to mesh topologies that utilize redundant paths to maximize availability in data centers.

With the rapid adoption of modern cloud and hybrid environments, traditional network boundaries have dissolved into complex virtual topologies.

This architectural shift creates significant visibility challenges, necessitating advanced monitoring engines (such as Snort and Suricata) capable of inspecting these intricate traffic flows.

2.1.3. Vulnerabilities

The relationship between cybersecurity and network design is deeply intertwined, as technical choices directly dictate a network's resilience against attacks.

Routers remain primary targets due to their external exposure, where neglected firmware updates or default credentials facilitate traffic interception or communication disruptions [16].

Switches, though internal, are vulnerable to MAC flooding attacks that force them to operate as hubs, increasing the risk of data interception when adequate network segmentation is absent.

Furthermore, wireless connections expand the attack surface, leaving access points susceptible to de-authentication and eavesdropping attacks if outdated protocols are used.

Endpoints represent the weakest link, often compromised via social engineering techniques like phishing, allowing attackers to establish a foothold for lateral movement.

Even robust foundational protocols like TCP/IP are susceptible to structural flaws, including IP spoofing and DNS cache poisoning attacks.

However, in modern enterprise and cloud environments, the vulnerability of network protocols extends far beyond traditional service disruptions to a more insidious threat: the exploitation of implicitly trusted diagnostic protocols. Standard perimeter firewalls and security groups inherently permit DNS (Port 53) and ICMP (Ping/Traceroute) traffic to maintain essential operational continuity and network reachability tests. Specifically, ICMP Echo Requests (Type 8) and Echo Replies (Type 0) rely on RFC 792 specifications that allow arbitrary, variable-length payload fields to test network throughput and MTU capacity. Because primary security controls treat diagnostic utility traffic as inherently

benign, they rarely subject ICMP payloads to Deep Packet Inspection (DPI). Adversaries exploit this architectural bypass by encapsulating unauthorized data streams and C2 instructions inside standard ICMP payloads, allowing covert exfiltration channels to pass through firewalls completely unmonitored. Consequently, this project evaluates the effectiveness of DPI engines—specifically Snort 3 and Suricata 8—in identifying these hidden tunnels within seemingly routine ICMP diagnostic traffic.

2.2. Types of IDSs

IDSs represent a critical layer of defense within a cybersecurity framework.

By monitoring network traffic and system activities for signs of malicious behavior or policy violations, IDSs provide organizations with the ability to detect and respond to threats in real time.

The classification of IDSs is a complex task, as different systems are designed to address specific operational environments, threat models, and performance requirements.

Broadly speaking, IDSs can be categorized into **NIDSs**, **HIDSs**, and **hybrid IDS**, with additional distinctions based on detection methodology, such as signature-based, anomaly-based, and specification-based systems.

• **NIDSs** operate at the network level, continuously analyzing packets traversing network links to identify malicious activity.

These systems are typically deployed at strategic points in the network infrastructure, such as at gateways or between subnets, to provide comprehensive visibility into traffic flows.

An **NIDS** relies on techniques like DPI and flow analysis to detect a wide range of threats, including denial-of-service (DoS) attacks, malware propagation, and port scanning.

One of the strengths of an **NIDS** lies in its ability to monitor multiple devices simultaneously, making it suitable for large-scale, distributed networks.

However, **NIDSs** face challenges in environments with high traffic volumes or encrypted data.

The performance of an **NIDS** can degrade under excessive traffic loads, leading to packet drops and missed detections.

Moreover, the increasing adoption of encryption protocols such as **TLS/SSL** complicates packet inspection, as the payload is inaccessible without decryption, potentially creating blind spots for the system.

Techniques such as decryption proxies or metadata analysis are employed to mitigate these challenges, but they introduce additional complexity and potential privacy concerns [16].

• **HIDSs** operate on individual endpoints, such as servers, desktops, or virtual machines, by monitoring system-level activities.

This includes observing log files, process behavior, file system modifications, and system calls.

An **HIDS** is well suited to detecting threats that specifically target the host, such as unauthorized access attempts, malware execution, or privilege escalation.

One of the primary advantages of **HIDSs** is their ability to detect local threats that might evade network-level monitoring.

For example, if a malicious actor gains access to a machine via a USB device or a phishing email, **HIDSs** can detect the subsequent unauthorized activities.

Additionally, **HIDSs** are particularly effective in environments where network visibility is limited, such as on encrypted endpoints or in cloud-based systems.

However, the reliance on individual host monitoring introduces scalability challenges in large organizations.

Managing, configuring, and maintaining **HIDSs** across hundreds or thousands of devices requires substantial administrative effort.

Moreover, **HIDSs** are inherently less capable of detecting attacks that span multiple hosts or the broader network, necessitating integration with network-level systems for a holistic defense.

Figure 1 shows schematic representation of the HIDS concept.

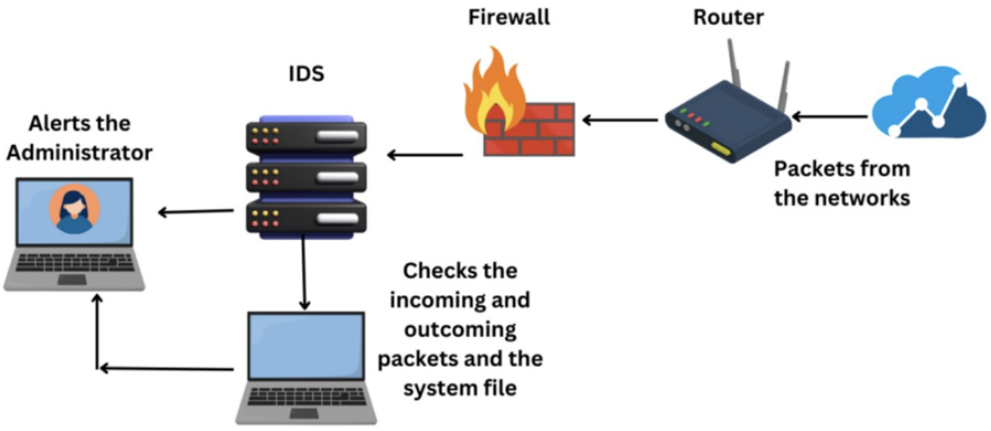


Figure 1: Schematic representation of the HIDS concept.

• **Hybrid IDSs** combine the capabilities of **NIDSs** and **HIDSs** to provide a more comprehensive security solution.

These systems leverage the strengths of both approaches by monitoring both network traffic and host activities.

By correlating data from multiple sources, **hybrid IDSs** can achieve greater accuracy in threat detection and reduce false positives.

A key advantage of hybrid IDSs is their ability to provide contextual aware ness.

For instance, a network anomaly detected by the **NIDS** can be correlated with suspicious file access on a host, enabling more effective detection of multi-stage attacks, such as APTs.

Furthermore, hybrid systems can provide defense-in-depth by applying detection mechanisms at both the network perimeter and individual endpoints.

However, the integration of multiple data streams introduces complexity in system design and operation.

Hybrid IDSs often require significant computational resources and sophisticated algorithms to process and correlate the vast amount of data generated.

The increased complexity also raises challenges related to deployment, configuration, and maintenance, as well as potential latency in real-time detection.

• **Signature-based IDSs** rely on predefined patterns or “signatures” of known threats to identify malicious activity.

These systems compare observed behavior or traffic against a database of signatures, such as specific byte sequences, known malware hashes, or anomalous commands.

Signature-based detection is highly effective for identifying well-documented threats and offers the advantage of low false-positive rates.

Despite their effectiveness against known threats, signature-based IDSs have significant limitations in detecting novel or polymorphic attacks.

Cyber adversaries frequently modify their techniques to evade detection, rendering signatures obsolete.

As a result, maintaining and updating the signature database is a continuous and resource-intensive process [18].

• **Anomaly-based IDSs** detect threats by identifying deviations from established base lines of normal behavior.

These baselines can be defined using statistical models, ML algorithms, or heuristic approaches.

Unlike signature-based systems, anomaly-based IDSs are capable of detecting previously unknown threats, making them suitable for dynamic and evolving threat landscapes.

However, the reliance on behavioral baselines introduces challenges, particularly in environments with high variability.

Normal behavior in such systems can be difficult to define, leading to an increased likelihood of false positives.

Furthermore, the computational overhead of anomaly detection is often higher than signature-based approaches, necessitating robust infrastructure and optimization to ensure real-time performance.

Figure 2 shows the schematic representation of both the anomaly-based and signature-based IDS concepts.

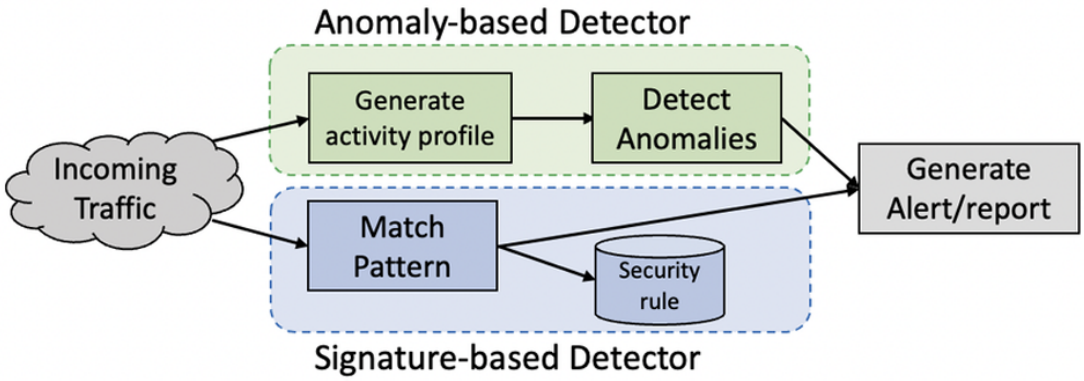


Figure 2: Schematic representation of both the anomaly-based and signature-based IDS concepts.

• **Specification-based IDSs** represent a hybrid approach that combines the deterministic nature of rule-based systems with the flexibility of anomaly detection.

These systems rely on predefined specifications of expected behavior, often created manually by experts, to identify violations indicative of malicious activity.

This approach offers higher precision compared to purely anomaly-based systems and is particularly effective in controlled environments with well-defined operational parameters, such as ICS or IoT networks.

However, the manual effort required to define specifications limits scalability and adaptability to new threats or rapidly changing environments.

The choice of IDS type depends on the operational environment, threat landscape, and resource constraints of the organization.

NIDSs are highly effective in perimeter defense but face challenges with encrypted traffic and high data volumes.

HIDSs excel in detecting endpoint-specific threats but require significant management effort in large deployments.

Hybrid IDSs offer the most comprehensive coverage but come at the cost of increased complexity.

Similarly, the choice between signature-based, anomaly-based, or specification based systems depends on the balance between detection accuracy, resource requirements, and adaptability to new threats.

2.3. IDSs Used

2.3.1. Snort

Snort is an open-source Network Intrusion Detection System (NIDS) developed initially in 1999.

Since then, it has become one of the most widely deployed intrusion detection and prevention technologies worldwide.

Snort captures and analyzes network traffic in real time, identifies potential threats, and alerts system administrators using a combination of signature-based detection, protocol analysis, and anomaly detection.

Snort's key features include the following:

1. Signature-Based Detection: Snort uses predefined rules to detect known threats by matching patterns in network traffic [19].

These rules are continuously updated by the community and Sourcefire (now part of Cisco) to include new threats as they emerge.

2. Protocol Analysis: Snort can analyze various network protocols to identify deviations from normal behavior that could indicate an attack.

This includes HTTP, FTP, TCP, UDP, and ICMP, among others.

3. Anomaly Detection: By establishing a baseline for normal network activity, Snort can identify unusual patterns that can indicate an attack.

4. Preprocessor Plugins: Snort's functionality can be extended via preprocessor plugins, which add new detection capabilities and enhancements to existing ones [19].

These preprocessors can reassemble fragmented packets, detect anomalies in HTTP requests, and decode various protocols.

5. Performance: Traditionally, Snort has been a single-threaded application, which could limit its performance on multi-core systems.

However, the Snort 3 release now includes a multi-threaded architecture, which improves its ability to handle high-throughput environments.

Snort is highly configurable, allowing users to customize its rules to suit their specific security needs.

It can be deployed in three main modes: **(1) Sniffer Mode:** In this mode, Snort captures and displays network packets in real-time.

(2) Packet Logger Mode, where it logs packets to disk, and **(3) NIDS Mode,** where it analyzes network traffic and generates alerts based on predefined rules.

Given these modes, Snort supports three deployment modes [19]; namely, **(1) real-time traffic analysis,** where Snort can monitor network traffic in real time and provide immediate alerts for suspicious activities;

(2) forensic analysis, where the captured traffic is used to analyze post-event to understand the nature and scope of an incident;

and **(3) regulatory compliance,** where it helps organizations meet various regulatory requirements by providing detailed logging and reporting capabilities.

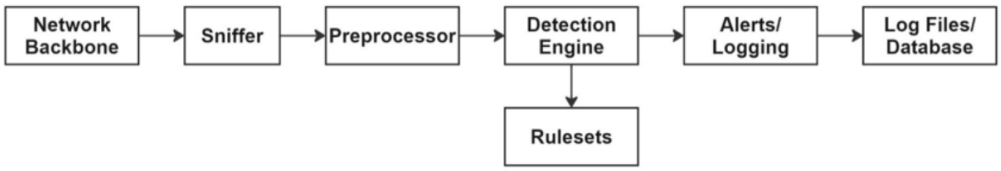


Figure 3: Snort Architecture

2.3.2. Suricata

Suricata is an open-source network threat detection engine developed by the Open Information Security Foundation (OISF) in 2009. It is designed to be a robust and high performance IDS/IPS and network security monitoring (NSM) tool [1].

Suricata leverages modern hardware capabilities, including multi-threading, GPU acceleration, and advanced protocol detection, making it a robust, scalable solution for detecting network-based threats [1].

Suricata's key features are summarized below [1]:

1. Multi-threading: Unlike Snort, which was traditionally single-threaded, Suricata is designed from the ground up to take advantage of multi-core processors.

This allows Suricata to handle higher traffic volumes and perform complex analyses more efficiently.

2. Protocol Identification and Parsing: Suricata includes advanced protocol identification and parsing capabilities that automatically detect and decode protocols such

as HTTP, TLS, FTP, and SMB.This feature enables deep packet inspection and accurate detection of protocol-specific attacks.

3. File Extraction: Suricata can extract and store files from network traffic for further analysis.

This is particularly useful for investigating malware infections and data exfiltration attempts.

Integration with External Tools: Suricata integrates seamlessly with other security tools and platforms.

It supports output to various formats and systems, including JSON, Elastic Stack, and Splunk, facilitating integration with Security Information and Event Management (SIEM) systems and other analysis tools.

4. Rules and Signatures: Suricata supports a rule syntax compatible with Snort, making it easy to transition from Snort to Suricata.

Furthermore, it supports advanced rule options and keyword extensions, enabling more granular detection.

5. Performance and Scalability: Suricata's architecture scales horizontally by adding more processing threads, enabling it to handle high-throughput environments.

It can leverage modern hardware features, such as CUDA and OpenCL, for GPU acceleration, further enhancing performance.

Suricata can be configured and deployed in various modes, similar to Snort [1]: **(1) NIDS Mode:** Monitor network traffic and generate alerts based on detection rules.

(2) IPS Mode: Acts on detected threats by dropping, rejecting, or modifying packets.

And **(3) NSM Mode:** Collects and logs detailed information about network traffic for analysis.

Suricata is highly configurable, with extensive options for tuning performance and detection capabilities.

It can be managed and configured using YAML files, which provide a straightforward and flexible way to define settings and rules.

Given these modes, Suricata is used in situations such as [19]: **(1) Real-time threat detection:** Suricata's multi-threaded design is suitable for environments with high traffic volumes, where it can provide real-time alerts for detected threats.

(2) Monitoring of network security: By capturing and analyzing detailed network traffic, Suricata helps organizations gain visibility into their network and detect potential security incidents.

And **(3) Threat hunting and forensic analysis:** Suricata's ability to extract files and log detailed traffic information makes it a valuable tool for post-incident analysis and threat hunting activities.

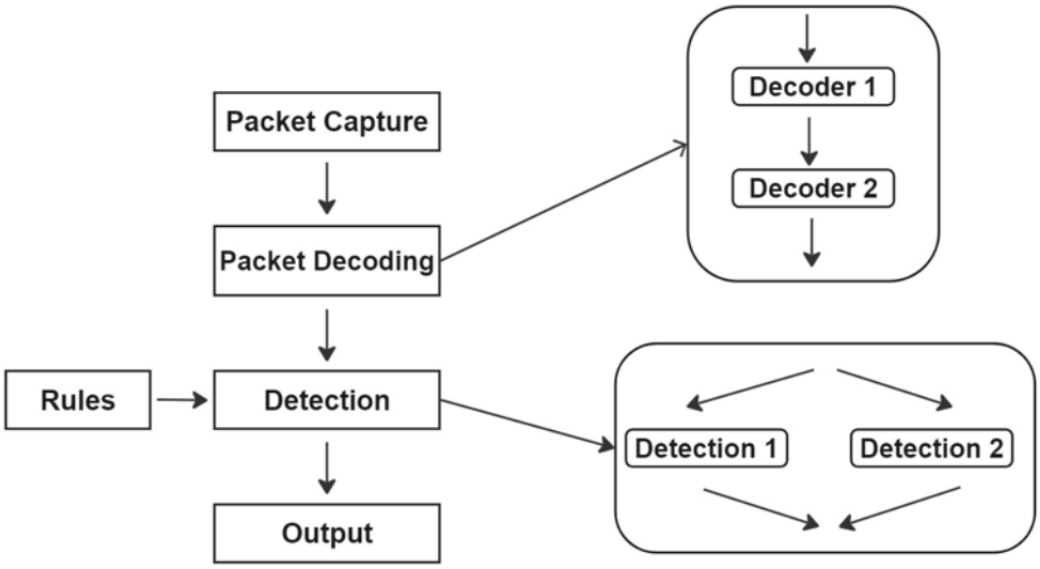


Figure 4: Suricata Architecture

2.4. Traffic Monitoring Mechanisms

2.4.1. Non-intrusive Monitoring Concept

The proposed security architecture adopts a non-intrusive monitoring methodology, where the Intrusion Detection System (IDS) is designed to operate as an out-of-path probe.

This approach enables deep packet inspection (DPI) of critical protocols without introducing latency or compromising the availability of the monitored physical processes.

By functioning as a transparent security layer, this architecture validates the integrity of data circulating between field sensors and control units, ensuring optimal detection accuracy while maintaining the strict real-time requirements of industrial operations.

2.4.2. On-Premise Traffic Capture Techniques: Network TAP vs. SPAN Port

To perform effective network security monitoring, an Intrusion Detection System (IDS) requires a reliable mechanism to capture data packets passing through the infrastructure.

In traditional on-premise environments, two primary hardware-layer methodologies are utilized: Network Test Access Points (TAPs) and Switched Port Analyzer (SPAN) ports, commonly known as Port Mirroring.

Switched Port Analyzer (SPAN) [15]: SPAN is a software-driven feature embedded within network switches that duplicates traffic from specific source ports or Virtual Local Area Networks (VLANs) and forwards it to a designated destination port connected to an IDS device.

Depending on the architectural scope, SPAN can be deployed as Local SPAN, where the source and destination ports reside on the same physical switch, or Remote SPAN (RSPAN), which routes mirrored traffic across multiple switches via a dedicated RSPAN VLAN.

For distributed geographical layers, Encapsulated Remote SPAN (ERSPAN) utilizes Generic Routing Encapsulation (GRE) to transport mirrored packets across Layer 3 domains.

Universally, a SPAN destination port operates in a unique operational state, typically appearing as UP/DOWN (monitoring), which prevents it from participating in Layer 2 protocols such as the Spanning Tree Protocol (STP) or Cisco Discovery Protocol (CDP), thereby mitigating network loops.

However, because SPAN relies on the switch's internal processing architecture—such as shared memory or supervisor engines—heavy traffic saturation can trigger oversubscription, leading the switch to drop mirrored packets in its output queues to prioritize production data traffic.

Furthermore, switches automatically discard corrupted frames at the physical layer, preventing the IDS from analyzing malformed packets.

Network Test Access Point (TAP): In contrast, a Network TAP is a passive, independent hardware device spliced directly into a physical network cable.

It clones network traffic at the physical layer (Layer 1) by splitting electrical or optical signals.

Because it operates independently of the network switch, a TAP introduces zero CPU overhead, cannot drop packets due to oversubscription, and captures all traffic, including

low-level physical errors and corrupted frames.

In this project, understanding these technical trade-offs is critical for choosing the optimal deployment model for the IDS probe.

It ensures that the selected data capture methodology provides total visibility without introducing a single point of failure or dropping critical packets during high-volume protocol anomalies.

2.4.3. Cloud Traffic Mirroring Mechanisms

The transition from traditional physical deployments to cloud-native architectures has fundamentally shifted how network traffic is captured. In virtualized cloud environments, physical access to network cables and hardware switches is entirely abstracted, rendering traditional TAPs and hardware SPAN configurations obsolete. To address this visibility gap, modern cloud service providers utilize Software-Defined Networking (SDN) to implement virtualized traffic capturing.

VPC Traffic Mirroring: [1] Native cloud solutions, such as AWS VPC Traffic Mirroring or Azure Virtual Network Traffic Mirroring, operate directly at the Hypervisor level or via the Virtual Network Interface Card (vNIC/ENI). This software-defined mechanism intercepts and copies inbound and outbound packets directly from the virtual interface before they reach the instance's operating system.The mirrored traffic is then encapsulated (typically using VXLAN headers) and securely routed across the cloud fabric to a centralized security monitoring instance. Since this research focuses on detecting covert channels within cloud environments, leveraging native cloud traffic mirroring allows the proposed IDS to analyze internal virtual networks seamlessly. This guarantees that deep packet inspection can be executed on cloud-hosted DNS and ICMP streams without requiring intrusive modifications to the production instances or altering the underlying virtual architecture.

2.4.4. Data Pipeline Architecture

An enterprise-grade intrusion detection framework cannot rely solely on localized packet capture; it demands a robust, structured data pipeline architecture to ingest, process, and transport telemetry seamlessly. This pipeline transforms raw network packets or localized engine logs into normalized, structured data suitable for centralized analysis and long-term storage.

A standard data pipeline consists of three core engineering phases [1]:

The Capture Phase: Raw network packets are mirrored using the aforementioned SPAN, TAP, or VPC mirroring techniques and delivered directly to the network interfaces of the

IDS sensor.

The Filtering and Normalization Phase: High-velocity raw traffic contains vast amounts of background data that can overwhelm processing units. In this stage, the IDS engine applies pre-capture and post-capture filtering to eliminate irrelevant protocol overhead. The engine parses the remaining packets, normalizes diverse protocol formats, and generates structured alert logs (such as JSON formats) that detail behavioral metrics.

The Transport Phase: To prevent local storage exhaustion and facilitate centralized monitoring, lightweight data shippers—such as Elastic Filebeat—are deployed. Filebeat acts as a resource-efficient agent that monitors the local IDS log directories, establishes an authenticated, encrypted connection, and ships the structured log data with fault-tolerant delivery guarantees to a centralized Security Information and Event Management (SIEM) system or an ELK stack analytics engine. The data pipeline designed for this project relies on these decoupled stages to ensure that alert logs and deep packet inspection data are securely transported to the centralized analysis engine.

This architecture prevents resource starvation on the monitored nodes, ensuring that security analysis remains completely decoupled from production processing.

2.4.5. Technical Monitoring Challenges: High Throughput and Encrypted Traffic

Deploying an effective network-based IDS in modern high-performance environments introduces significant engineering challenges that threaten detection accuracy and system stability.

The two most critical bottlenecks are packet drop under massive throughput and protocol visibility loss due to widespread encryption.

High Throughput and Packet Drops: Modern enterprise and cloud data centers routinely operate at gigabit and terabit speeds.

When an IDS face high-throughput traffic, its central processing unit can become a bottleneck.

Traditional, single-threaded IDS architectures fail to scale across multi-core processors, resulting in extensive packet drops where the engine simply misses chunks of traffic, creating critical blind spots for security teams.

To overcome this, modern engines must utilize advanced multi-threading architectures (such as Suricata's multi-threaded pipeline or Snort 3's modern worker threads) [1] alongside zero-copy packet capture frameworks like PF_RING or AF_PACKET to balance the traffic load symmetrically across multiple CPU cores.

The Encrypted Traffic Blind Spot: The near-universal adoption of robust cryptographic protocols like TLS 1.3 and HTTPS ensures data privacy but severely hinders network security monitoring.

Because payload contents are heavily encrypted, traditional Deep Packet Inspection (DPI) engines cannot read the packet data to match known attack signatures.

To mitigate this challenge without compromising data privacy through complex, resource-heavy SSL/TLS Decryption Offloading architectures, modern detection strategies must adapt.

This requires moving away from pure payload inspections and focusing instead on unencrypted cryptographic metadata, handshake analysis (such as JA3 fingerprinting), and behavioral traffic analysis to identify anomalies [1].

This project directly addresses these challenges by evaluating multi-threaded IDS engines to prevent packet drops under high-throughput conditions, while focusing specifically on protocol metadata and behavioral anomalies.

This approach allows the system to identify covert tunneling activities within DNS and ICMP traffic without requiring intrusive payload decryption.

2.5. Test Instrumentation: The ELK Stack

To analyze IDS output, the Elasticsearch, Logstash, and Kibana (ELK) stack is used for log processing and visualization.

In addition, DNS tunneling is examined since it is a critical exfiltration technique relevant to the study.

The ELK Stack, comprising Elasticsearch, Logstash, and Kibana, is a log analysis platform used for IDS monitoring.

It is widely used to search, analyze, and visualize huge volumes of log data in real time, making it an essential tool for monitoring and analyzing logs generated by IDS tools such as Snort and Suricata.

The key features of the ELK stack are [20]:

Elasticsearch: Elasticsearch is a distributed, RESTful search and analytics engine.

It is designed to handle large-scale data ingestion and retrieval, enabling users to store, search, and analyze big data quickly and in near real-time.

Elasticsearch utilizes a schema-free JSON document structure to index data, making it flexible and scalable. Elasticsearch's key features include [21]:

• Full-text search capabilities.

• Near real-time indexing and search.

• Distributed and horizontally scalable architecture.

• RESTful API for seamless integration with other tools and platforms.

Logstash: Logstash is a server-side data processing pipeline that ingests data from various sources, transforms it, and sends it to the desired output.

It supports various input sources, including logs, metrics, web applications, data stores, and various AWS services.

Logstash uses a rich ecosystem of plugins to parse and transform data, ensuring that it is structured and formatted correctly before indexing to Elasticsearch.

Key features include:

• Centralized data processing with support for custom log formats.

• Extensible architecture with numerous plugins for the input, filter, and output stages.

• Ability to parse and transform unstructured data into structured formats.

Kibana: Kibana is a data visualization and exploration tool that sits on top of Elasticsearch, providing an intuitive web interface for analyzing and visualizing data.

It allows users to create and share dynamic dashboards that reflect real-time data changes.

Kibana supports a variety of visualizations, including histograms, line graphs, pie charts, and geographic maps. Key features include:

• Real-time visualization and analysis of data stored in Elasticsearch.

• User-friendly interface to create and share dashboards.

• Advanced query and filtering capabilities using the Kibana Query Language (KQL).

• Support for alerting and reporting based on data insights. The ELK Stack could be integrated with Filebeat.

Filebeat: Filebeat is a lightweight shipper that centralizes log data [22].

Installed on your servers, Filebeat monitors the log files or locations you specify, collects log events, and forwards them to Logstash or Elasticsearch for indexing.

When integrated with the ELK Stack, Filebeat simplifies the collection, parsing, and visualization of logs from IDS tools such as Snort and Suricata [23].

Given these modes, the ELK Stack could be used insituations such as [23]: **(1) Real-time Monitoring:** The ELK Stack enables real-time monitoring of log data, allowing immediate detection and response to security incidents.

(2) Threat Analysis and Visualization: Kibana's visualization capabilities help security teams identify patterns and anomalies in network traffic, aiding in threat analysis and investigation.

And **(3) Compliance and Reporting:** ELK Stack can generate reports for compliance purposes and provide detailed logs of network activities and security events.

2.6. Attack and Tunneling Mechanisms

2.6.1. DNS Tunneling: DNSCat2 and Iodine

Given the increasing sophistication of cyber threats, it is crucial to understand the methods attackers use to bypass conventional security mechanisms.

One such method is Domain Name System (DNS) tunneling. DNS tunneling is a technique that encapsulates data within DNS packets [24].

DNS, which is specially designed to translate domain names into IP addresses, is often allowed through firewalls and network security systems because it is essential for normal Internet operations. This makes DNS an attractive vector for attackers to bypass traditional security measures and exfiltrate data or establish covert communication channels.

DNS tunneling exploits the DNS protocol by embedding data in DNS queries and responses, creating a covert channel for data exfiltration.

In DNS tunneling, an attacker creates a rogue DNS server that communicates with malware on the victim's machine. The malware encodes data in the DNS query's subdomain, which is then sent to the rogue DNS server. This server decodes the data and processes them accordingly. Figure 5 provides a visual explanation of the technique. This method is effective because DNS traffic is generally permitted by firewalls and is not subject to the same rigorous scrutiny as other traffic.

Various tools, such as Iodine (https://github.com/yarrick/iodine) and DNScat2 (https://github.com/iagox86/dnscat2), facilitate DNS tunneling by encapsulating data

within DNS packets. These tools establish an IP-over-DNS tunnel, enabling data transmission over DNS. This technique can be used not only for data exfiltration, but also for establishing C2 channels, making it a versatile tool for attackers [25,26].

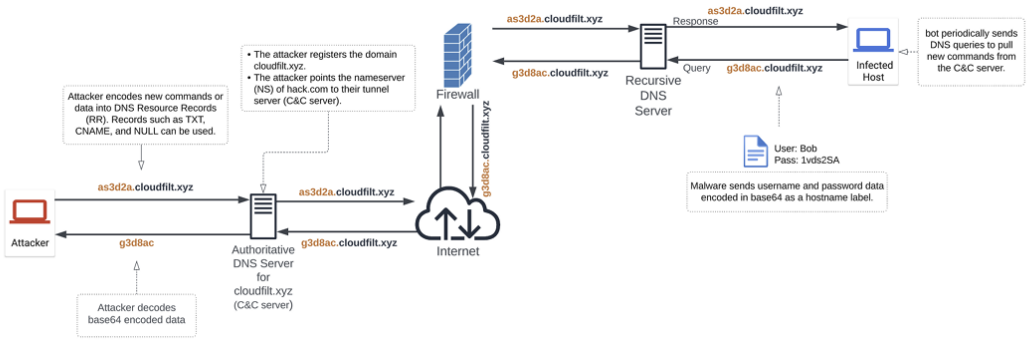


Figure 5: Illustration of DNS Tunneling Used for C2 Communication.

DNSCat2: A tool used to encapsulate data inside DNS subdomains, leveraging standard query records and responses like TXT and CNAME [1].

Iodine: A tool designed to establish an IP-over-DNS tunnel, which specifically exploits the NULL resource record type to transmit raw binary data seamlessly [1].

2.6.2. ICMP Tunneling (Metasploit):

ICMP tunneling is a covert communication mechanism that encapsulates arbitrary transport-layer payloads (such as TCP or UDP traffic) inside standard Internet Control Message Protocol packets [10]. According to RFC 792 specifications, Type 8 (Echo Request) and Type 0 (Echo Reply) packets accommodate a variable-length data field designed for network latency and reachability diagnostics. Because the protocol specification does not strictly constrain or validate the contents of this payload field, adversaries exploit this flexibility to establish bi-directional covert channels that bypass traditional boundary firewalls [10].

Technically, a specialized tunneling agent intercepts application traffic and embeds it into the ICMP data section [10]. To support concurrent connections, tunneling tools leverage the 16-bit Identifier and Sequence Number fields within the ICMP header to achieve session multiplexing. Accounting for standard Ethernet Maximum Transmission Unit (MTU = 1500 bytes), IP header (20 bytes), ICMP header (8 bytes), and TCP header (20 bytes), an ICMP tunnel can transmit up to 1452 bytes of encapsulated TCP payload per frame, introducing minimal overhead compared to standard communications [10].

Real-world implementations range from open-source penetration testing utilities such as Ptunnel and Metasploit's icmp_tunnel module to sophisticated malware frameworks like

the Pingback backdoor and Cobalt Strike beaconing mechanisms. Because network administrators rarely inspect ICMP payloads beyond basic reachability tests, these covert channels represent a significant threat vector for undetected data exfiltration and Command-and-Control (C2) operations [10].

2.7. Virtualization Technology

Virtualization is a software-based technology that abstracts physical hardware resources—such as the CPU, memory, storage, and network interfaces—into multiple distinct, isolated virtual components. This abstraction layer enables the concurrent execution of multiple independent operating systems, known as Virtual Machines (VMs), on a single physical hardware host. The allocation, management, and abstraction of these virtualized resources are governed by a specialized software layer termed the Hypervisor or Virtual Machine Monitor (VMM).

Hypervisors are architecturally classified into two primary categories:

Type-1 (Bare-Metal): Hypervisors that execute directly on the underlying bare hardware without an intermediate operating system, delivering high throughput, low latency, and enterprise-grade efficiency (e.g., VMware ESXi).

Type-2 (Hosted): Hypervisors that operate as software applications on top of an existing host operating system, making them highly suitable for localized testing and modular laboratory environments (e.g., VMware Workstation).

In the context of this research, virtualization serves as the foundational infrastructure required to safely simulate cyber-attack and defense paradigms.

By provisioning a strictly isolated, sandboxed environment, it allows the concurrent deployment of the attacker node, the victim node, and the monitoring intrusion detection system (IDS) on a single physical machine, thereby mitigating any operational security risks to external production networks during live exploitation.

2.7.1. VMware Virtualization Platform

VMware is an industry-standard enterprise platform specializing in virtualization and cloud computing architectures. It provides advanced virtual networking components that are critical for complex security research, traffic interception, and granular network analysis.

Key technical capabilities within the VMware ecosystem include:

Virtual Switches (vSwitches): Software-defined components that logically replicate the functionality of physical switches, allowing the configuration of isolated internal network

segments (such as Host-Only or NAT topologies).

Snapshots: A feature that preserves the exact state and memory profile of a virtual machine at a specific point in time, enabling seamless and instant restoration during iterative malware or attack testing.

Promiscuous Mode: A critical security and networking configuration within the vSwitch policies.

When enabled, it permits a virtual Network Interface Card (vNIC) assigned to a specific instance to capture, read, and intercept all data packets traversing the virtual switch, completely bypassing standard destination MAC address filtering.

The VMware platform was selected as the deployment environment for this project primarily due to its robust and deterministic implementation of Promiscuous Mode across its virtual switching fabric.

This capability is a mandatory technical prerequisite that empowers the deployed Network Intrusion Detection System (NIDS) to sniff, intercept, and perform Deep Packet Inspection (DPI) on all mirrored traffic exchanged between the malicious and compromised instances, ensuring the successful identification of covert data exfiltration tunnels (such as DNS and ICMP channels).

2.8. Evaluation Metrics

Performance metrics are calculated based on the detection rates and accuracy of each IDS.

We have used the following metrics to evaluate the performance of the IDS tools.

• **True Positive (TP):** represents the number of correctly identified attacks.

• **False Positive (FP):** represents the number of legitimate activities incorrectly identified as attacks.

• **False Negative (FN):** represents the number of attacks that were not identified.

• **True Negative (TN):** represents the number of legitimate activities correctly identified as non-attacks.

From the aforementioned values, we then calculate the following performance metrics:

• **Recall (True Positive Rate (TPR)):** Measures the ability to identify actual attacks and is calculated as Equation (1):

(1)

$$\text{Recall} = \frac{TP}{TP + FN}$$

• **Precision:** Measuresthe accuracy of positive predictions and is calculated as Equation (2):

$$\text{Precision} = \frac{TP}{TP + FP} \tag{2}$$

• **Accuracy:** Measures the model's overall accuracy and is calculated as in Equation (3):

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN} \tag{3}$$

Since True Negatives (TN) are not applicable in our scenarios, we simplify accuracy to be calculated as Equation (4):

$$\text{Accuracy} = \frac{TP}{TP + FP + FN} \tag{4}$$

• **F1-Score:** The harmonic mean of precision and recall provides a metric that balances both concerns and is calculated as Equation (5):

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times Recall}{\text{Precision} + Recall} \tag{5}$$

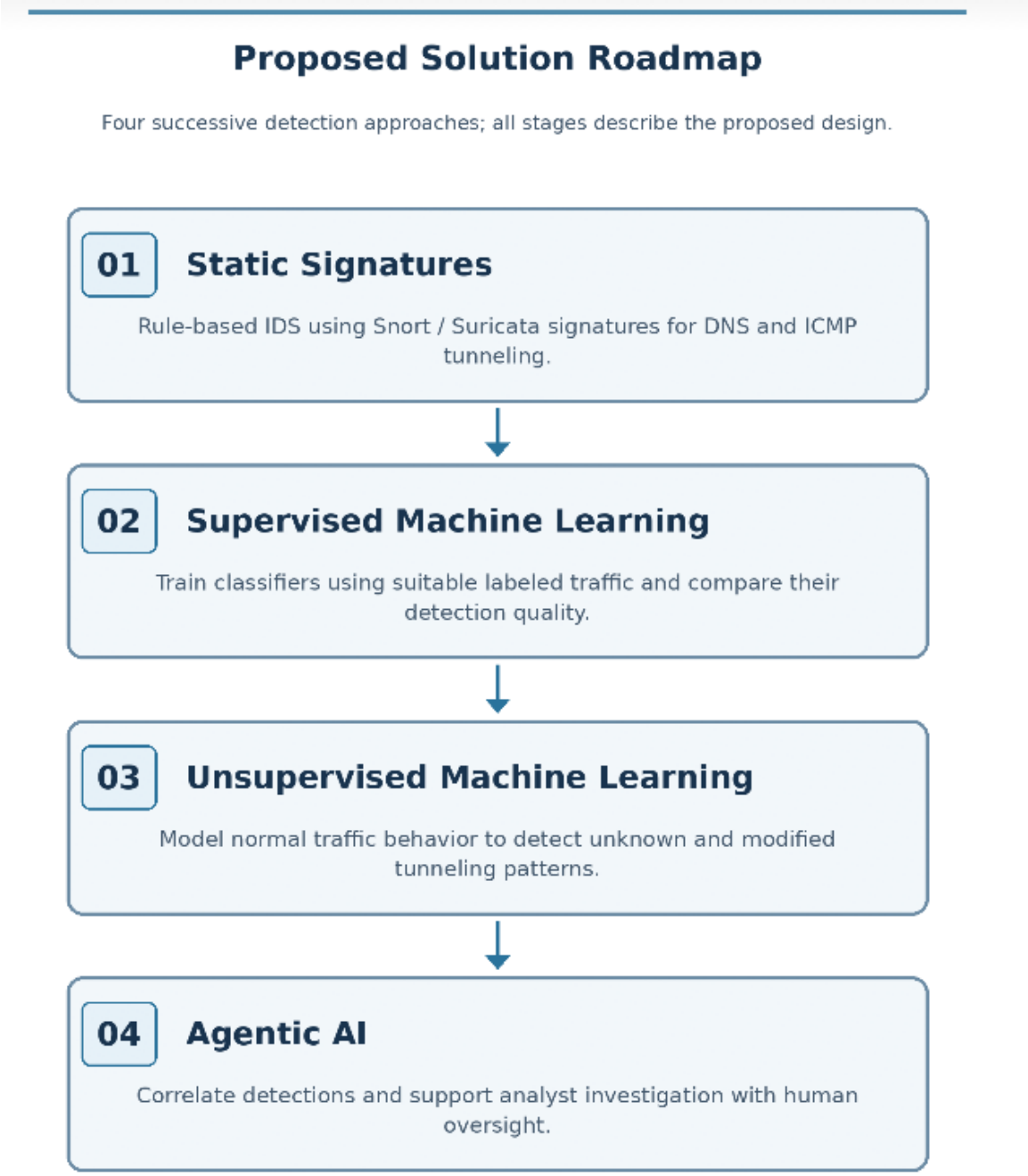


Figure 6: Proposed Solution Roadmap.



Figure 7: Proposed Implementation Flowchart.

References

1. A Comparative Evaluation of Snort and Suricata for Detecting Data Exfiltration Tunnels in Cloud Environments. J. Cybersecur. Priv. 2026, 6, 17 [CrossRef]

2. Martínez, J.; Durán, J.M. Software supply chain attacks, a threat to global cybersecurity: SolarWinds' case study. Int. J. Saf. Secur. Eng. 2021, 11, 537–545. [CrossRef]

3. Adiwal, S.; Rajendran, B.; Shetty D., P.; Sudarsan, S. DNS Intrusion Detection (DID)—A SNORT-based solution to detect DNS Amplification and DNS Tunneling attacks. Frankl. Open 2023, 2, 100010. [CrossRef]

4. Hari, D.;Raharjo, K.; Salman, M. Analyzing Suricata Alert Detection Performance Issues Based on Active Indicator of Compromise Rules. J. Tek. Inform. (Jutif) 2023, 4, 601–610. [CrossRef]

5. V. A. Phan and J. Jerabek, "Evasive IPv6 Covert Channels: Design, Machine Learning Detection, and Explainable AI Evaluation," in *Proceedings of the 22nd International Conference on Security and Cryptography (SECRYPT 2025)*, 2025, pp. 666–675.

6. G. Kaddissi, M. Hatoum, K. Samrouth, and A. El Attar, "ICMP Covert Channel Ranking System (ICMP C2RS) for Real-Time Threat Identification," in *Proceedings of the 5th IEEE Middle East and North Africa Communications Conference (MENACOMM 2025)*, IEEE, 2025, pp. 112–118.

7 S. Lipsa, S. R. Mohanty, and A. Sahoo, "An interpretable dimensional reduction technique with an explainable intrusion detection system for ICMP tunneling," PMC Computer Science, vol. 14, pp. 102–118, 2025.

8. M. Ahmad, R. Al-Dmour, and S. Hassan, "Covert Channel Detection: Machine Learning Approaches and Evaluation in Modern IP Networks," *IEEE Access*, vol. 13, pp. 14200–14215, 2025.

9. A. Al-Ahmad and N. Huda, "A Multi-Layer Network Steganography Framework Using ICMP and NTFS ADS," *International Journal of Emerging Technologies in Security*, vol. 18, no. 1, pp. 45–58, Jan. 2026.

10. G. Merezas and L. Wüstrich, "Covert Communication over ICMP," Seminar IITM, Chair of Network Architectures and Services, School of Computation, Information and Technology, Technical University of Munich (TUM), 2024.

11. R. Kumar and P. Singh, "Stateless Scrubbing Defenses Against Malicious ICMP Tunneling in Enterprise Networks," *Computers & Security*, vol. 148, p. 103850, 2025.

12. H. Liu, J. Zhao, and M. Sun, "A control-plane analysis framework for identifying anomalous ICMP communication patterns," *IEEE Communications Letters*, vol. 29, no. 2, pp. 312–316, 2025.

13. C. Wang, Z. Li, and T. Zhou, "A baseline feature-based malicious traffic detection method for Internet Control Message Protocol (ICMP) covert tunnel," *Journal of Cybersecurity and Privacy*, vol. 8, no. 1, pp. 22–37, 2025.

14. Y. Chen, K. Xu, and B. Yang, "CC-Guard: An IPv6 Covert Channel Detection Method Based on Field Matching," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 1890–1902, 2025.

15. Network Analysis of Students' Online Activities via Port mirroring Switch Port Analyzer. IEEE 13 June 2022 . [CrossRef]

16. Aslan, Ö.; Aktu˘g, S.S.; Ozkan-Okay, M.; Yilmaz, A.A.; Akin, E. A comprehensive review of cyber security vulnerabilities, threats, attacks, and solutions. Electronics 2023, 12, 1333. [CrossRef]

17. Ring, M.; Wunderlich, S.; Scheuring, D.; Landes, D.; Hotho, A. A survey of network-based intrusion detection data sets. Comput. Secur. 2019, 86, 147–167. [CrossRef]

18. Kwon,H.Y.; Kim, T.; Lee, M.K. Advanced intrusion detection combining signature-based and behavior-based detection methods. Electronics 2022, 11, 867. [CrossRef]

19. Boukebous, A.; Fettache, M.; Bendiab, G.; Shiaeles, S. A comparative analysis of Snort 3 and Suricata. In Proceedings of the IEEE IAS Global Conference on Emerging Technologies (GlobConET), London, UK, 19–21 May 2023; pp. 1–6. [CrossRef]

20. Mitra, M.; Sy, D. The Rise of Elastic Stack. 2016. Available online: https://www.researchgate.net/publication/309732494_The_Rise_of_Elastic_Stack (accessed on 20 December 2025).

21. Kathare, N.; Reddy, O.; Prabhu, V. A comprehensive study of elasticsearch. J. Res. Sci. Eng. 2022, 4 , 716–720. [CrossRef]

22. Orsos, M.; Torok, R.; Farago, C.; Antalfia, B.; Kail, E.; Banati, A. Security Operation Center Methodology for 5G Networks. Acta Polytech. Hung. 2025, 22, 99–121. [CrossRef]

23. Bavaskar, P.; Kemker, O.; Sinha, A.; Sabri, M. A Survey On: 'Log Analysis With Elk Stack Tool'. Int. J. Res. Anal. Rev. 2019, 6, 965–968.

24. Asani, E.; Ayoola, M.; Aderemi, E.; Adedayo-Ajayi, V.; Ayoola, J.; Akande, O.; Adeniyi, J.; Olowe, O.; Ibitoye, N. L2R-MLP: A Multilabel Classification Scheme for the Detection of DNS Tunneling. Data Sci. Manag. 2024, 1, 323–331. [CrossRef]

25. Sani, F.; Setiawan, M. DNS tunneling Detection Using Elasticsearch. IOP Conf. Ser. Mater. Sci. Eng. 2020, 722, 012064. [CrossRef]

26. Salat, L.; Davis, M.; Khan, N. DNS Tunnelling, Exfiltration and Detection over Cloud Environments. Sensors 2023, 23, 2760. [CrossRef]